

Adaptive temporal subsampling for Zakharov rollouts

Event-aware per-case schedules for Tanaka collisions and Benjamin–Feir recurrence

11 May 2026

Executive summary

Production data is generated by integrating the Zakharov equations on a dense time grid ($\Delta t = 0.08$, $t_{\max} = 200 \Rightarrow T = 2501$ steps) and saving $K = 200$ snapshots per case for FNO training. Uniform- K subsampling spreads those 200 snapshots evenly across the trajectory, but the events that actually stress the surrogate — soliton collisions in Tanaka, envelope recurrence in Benjamin–Feir — are short and locally complicated. With uniform sampling those events fall between snapshots and are under-represented in training. The fix is a per-case schedule that concentrates samples at events while preserving full-trajectory coverage. Schedules are deterministic (inverse-CDF over a quantile grid; no RNG), per-case (each trajectory picks its own indices), and add negligible cost on top of the existing rollout.

1 Motivation: why uniform fails

1.1 Conserved quantities are useless as event detectors

The Zakharov Hamiltonian is conserved along trajectories, so any total-energy diagnostic is flat in time and carries no event information. We need a *non-conserved* scalar that spikes during the focusing / collision / recurrence events we care about.

1.2 FNO state sensitivity

Empirically, FNO rollouts diverge fastest at the moments where the underlying PDE state changes rapidly — soliton overlap, BF envelope peaks. These regions are exactly where the training mix is thinnest under uniform- K sampling: a Tanaka collision is $\mathcal{O}(1)$ wide on a 200s trajectory, so uniform sampling lands maybe 1 sample on it; BF recurrence cycles span ~ 50 s and again get under-sampled at $K = 200$.

2 Activity signal

2.1 Two regime-specific choices

We use two different non-conserved signals depending on the regime:

- **Tanaka (soliton collisions).** Surface-gradient energy

$$S(t) = \|\eta_x(t)\|^2 = \int_0^L \eta_x(x, t)^2 dx,$$

computed via spectral derivative. This dips during destructive tail overlap before a collision (since the inner product $\int \eta_{1,x} \eta_{2,x} dx$ becomes negative as solitons of opposite orientation overlap) and spikes at peak focusing. The dip-spike pattern means $|dS/dt|$ is large on *both* sides of the collision, which is what we want for biasing samples toward the event.

- **Benjamin–Feir (envelope recurrence).** Envelope peak

$$M(t) = \|\eta(t)\|_\infty = \max_x |\eta(x, t)|.$$

Surface-gradient energy is dominated by the fast carrier and washes out the slow envelope modulation; the envelope peak tracks recurrence directly, with $M(t)$ humping at each Akhmediev focusing.

Both signals are scalar per (case, time), computed on-device from the (T, B, N) eta trajectory just after the rollout finishes.

2.2 Memory-aware computation

The naive spectral derivative for $S(t)$ takes a full (T, B, N) complex FFT intermediate, which at $T = 2501$, $B = 256$, $N = 1024$ in float64 is ~ 20 GB and OOMs on a 48 GB card alongside the eta/xi buffers. We compute it instead with `jax.lax.scan` over the time axis so the FFT intermediate is only (B, N) per step:

```
@jax.jit
def grad_energy_traj(eta_traj, length):
    n = eta_traj.shape[-1]
    k = 2.0 * jnp.pi * jnp.fft.fftfreq(n, d=length / n)
    def step(_, eta_t):
        eta_hat = jnp.fft.fft(eta_t, axis=-1)
        eta_x = jnp.real(jnp.fft.ifft(1j * k * eta_hat, axis=-1))
        return None, jnp.sum(eta_x ** 2, axis=-1)
    _, s_TB = jax.lax.scan(step, None, eta_traj)
    return jnp.swapaxes(s_TB, 0, 1) # (B, T)
```

The envelope-peak signal is just `jnp.max(jnp.abs(...), axis=-1)` and needs no scan.

3 Density construction

For each case b we build a sampling density over the dense time grid $\{t_0, \dots, t_{T-1}\}$:

$$\text{density}_b(t) = (1 - \alpha) \text{uniform}(t) + \alpha \text{smooth}(\text{activity}_b(t)),$$

where $\alpha \in [0, 1]$ mixes a uniform prior (always covers the full trajectory) against an activity-weighted bias. The smoothing is Gaussian with width σ (in dense-time steps) to avoid spiky over-concentration on a single step.

Two ways to build activity. We provide two modes for turning a raw signal into a density weight:

- **density_mode = "rate":** $\text{activity}(t) = |dS/dt|/(S + \varepsilon)$. Picks up localized state changes; correct for Tanaka collisions, which are short bursts on top of a quiet baseline.

- **density_mode = "magnitude":** $\text{activity}(t) = S(t) - \min_t S(t)$. Picks up where the signal is high; correct for BF, where the envelope modulates continuously and we want to bias toward whichever times have the largest peaks rather than the moments of fastest change.

Power sharpening. For shallow modulation (BF envelope typically only moves $\sim 20\%$ from trough to peak), the magnitude density is nearly uniform on its own. We sharpen by raising it to a power:

$$\text{activity}(t) \mapsto (\text{activity}(t))^p,$$

applied after subtracting the per-case minimum (so the baseline doesn't dominate after exponentiation). For the BF regen $p = 4$ gives clear three-cluster concentration on the envelope peaks; $p = 1$ stays flat.

4 Deterministic inverse-CDF sampling

Given the per-case density, we pick K indices via inverse-CDF at K equispaced quantiles:

$$\text{cdf}(t) = \text{cumsum}(\text{density} / \sum \text{density}), \quad (1)$$

$$q_k = (k + \frac{1}{2})/K, \quad k = 0, \dots, K-1, \quad (2)$$

$$\text{idx}_k = \text{searchsorted}(\text{cdf}, q_k). \quad (3)$$

We then pin endpoints ($\text{idx}_0 = 0, \text{idx}_{K-1} = T - 1$) and sweep once to enforce strict monotonicity in case adjacent quantiles land on the same index. No RNG: same trajectory always produces the same indices, and two cases with identical activity profiles get identical schedules.

The result is an (B, K) `int32` array of per-case indices into the dense time grid.

5 Plumbing into data generation

5.1 End-to-end per-batch flow

For each batch of $B = 256$ cases:

1. **Rollout.** `batched_rollout` on the dense grid produces `eta_TBN` and `xi_TBN` of shape (T, B, N) , kept on-device.
2. **Activity signal.** One of `grad_energy_traj` (scanned over time) or `envelope_peak_traj` reduces `eta` to a (B, T) scalar field. Transferred to host.
3. **Index selection.** `adaptive_indices_from_signal` on host builds the per-case density, integrates the CDF, and picks $\text{idx} \in \mathbb{Z}^{B \times K}$.
4. **Gather.** `jax.numpy.take_along_axis` slices `eta` and `xi` at the per-case indices, producing (B, K, N) tensors.
5. **Sparse gxi.** $G(\eta)\xi$ is computed via spectral DNO series only at the K selected times (not the dense T), in chunks to keep memory bounded. This is a major win: the dense-grid `gxi` pass would cost $T/K \approx 12\times$ more.
6. **Flatten and write.** The (B, K, N) tensors flatten to (BK, N) for the `npz`, alongside the (B, K) index array and the dense-time vector so each sample's actual time is recoverable.

5.2 Why per-case schedules

A single shared schedule (the same K time indices for every case in a batch) would be uniform from each case’s point of view, because different cases peak at different times: a 2-soliton collision happens at the case-specific time when the solitons overlap, BF recurrence period scales with $1/\varepsilon^2$ per case. Per-case scheduling adds nothing to the runtime budget — one tiny `searchsorted` per case after the rollout — and is the whole point.

6 Parameter choices for the regen runs

	Tanaka	BF
signal	$\ \eta_x\ ^2$ (grad_energy)	$\ \eta\ _\infty$ (envelope)
density mode	rate	magnitude
α	0.5	0.85
σ (steps)	50	20
power	1.0	4.0

Table 1: Adaptive sampling parameters used in the production regeneration of `tanaka_2_adaptive_*` and `bf_2_adaptive_*`.

Why α differs. $\alpha = 0.5$ on Tanaka leaves substantial uniform coverage because much of the trajectory between collisions is still worth sampling (the FNO sees plenty of clean propagation, but collisions are where it fails). $\alpha = 0.85$ on BF concentrates more aggressively on recurrence peaks because the envelope modulation *is* the only structure to lock onto — there’s no quiet baseline to subsidize uniformly.

Why σ differs. $\sigma = 50$ on Tanaka smooths the $|dS/dt|/S$ rate signal over ~ 4 s of dense-grid time, broadening the collision peak so we sample its approach and aftermath, not just the apex. $\sigma = 20$ on BF is enough because the envelope humps are themselves slow features that don’t need much smoothing.

7 Smoke-test results

The adaptive sampler is implemented in `solver/gen_data/adaptive_sampling.py` and exercised by two demos:

- `solver/evals/adaptive_subsample_demo.py` — 2-crest Tanaka at $h = 0.05$. Renders the $\eta(x, t)$ heatmap with both uniform and adaptive sample times overlaid, plus the actual saved (η, ξ) at each adaptive snapshot. Confirms that the rate density clusters samples around the collision event and leaves uniform coverage of the propagating regime.
- `solver/evals/adaptive_subsample_demo_bf.py` — canonical BF setup ($n_{\text{carr}} = 10$, $\varepsilon_c = 0.12$, $\text{side_offset} = 3$, $h = 1.5$, $t_{\text{max}} = 150$). Confirms that the magnitude density with $\alpha = 0.85$, $p = 4$ produces three sample clusters aligned with the three envelope-peak recurrence events visible in $\|\eta\|_\infty(t)$, whereas $p = 1$ produces nearly uniform samples because the modulation only spans $\sim 20\%$.

8 Generator CLI

Both `generate_tanaka_dataset_v2` and `generate_bf_dataset` accept the following flags:

```
--adaptive                                # opt in
--adaptive_signal {envelope, grad_energy}
--adaptive_density_mode {rate, magnitude}
--adaptive_alpha <float in [0, 1]>
--adaptive_smooth_sigma <float>          # Gaussian sigma in dense-time steps
--adaptive_power <float>                  # 1.0 = none; raise for sharper bias
```

Without `--adaptive`, the generator falls back to the existing uniform stride logic. With `--adaptive`, the per-batch index array is also written to the npz under `subsample_indices_{batch_tag}.npz` so each saved sample's source dense-grid time is fully recoverable post-hoc.

9 Cost

The adaptive machinery adds, per batch:

- One `scan` reduction of η over T steps (`grad_energy` path) or one `max` reduction (`envelope` path) — both well under 1 s on A100 / RTX 6000 Ada at $T = 2501$, $B = 256$, $N = 1024$.
- One host-side density+CDF+searchsorted per case — microseconds.
- Gather + sparse gxi pass replacing the dense gxi pass — this is a *savings*, not a cost, since the dense pass evaluated $G(\eta)\xi$ at all T time slices.

Net effect on wall time per batch: zero to mildly negative.